Technical University of Applied Sciences Würzburg-Schweinfurt (THWS)

Backwarden

Portfolio 04: Documentation

Christian Thorwarth (5124022) · Luke Behrsing (5124075) · Simon Hümmer (5124021)

1 Introduction

Our project is a secure password management API that allows users to store, organize, and retrieve their credentials safely, ensuring that sensitive login data never gets lost or exposed.

1.1 Domain Summary

In today's digital world, users handle dozens of online accounts, each requiring strong, unique passwords. Remembering all these credentials is nearly impossible, leading many users to reuse weak passwords or store them insecurely. This creates major security risks, especially for small teams and individuals who lack professional password management tools. Our project addresses this issue by providing a centralized backend service for password storage and retrieval. Each user can create and manage multiple entries, each containing login data (e.g., website, username, password, notes). The system ensures data integrity and allows easy CRUD operations.

1.2 Key Use Cases

- The Secure User Onboarding process serves as the entry point to the system. It handles the registration of new users and the establishment of their security context. The focus of this use case is on enforcing strict password policies ensuring that every user starts with a valid and secure state.
- Representing the core value of the application, Encrypted Credential Management allows users to store and retrieve login information. The implementation demonstrates the separation of sensitive data from persistence: the business logic ensures that passwords are encrypted using AES before they ever reach the database adapter, guaranteeing that plaintext data is never stored.
- To manage the organization of secrets, the Hierarchical Data Lifecycle enables users to delete vaults. This use case illustrates the domain's compositional logic. It ensures data consistency by enforcing a cascading deletion: when a vault is removed, all associated credentials are automatically and permanently destroyed to prevent orphaned or insecure data residues.

2 Software Architecture

The core architectural choice for this project was the Hexagonal Architecture (also known as Ports and Adapters). The primary goal of this pattern is to create a clear separation between the domain logic and the technical infrastructure. In our implementation, the domain remains the "inside" of the hexagon, completely agnostic of external technologies like databases, Quarkus, or communication protocols (REST). This decoupling ensures that the business logic is independently testable and protected from the volatile nature of external dependencies.

2.1 The Domain Layer: Core Business Logic

- **Models**: Our core models `User`, `Vault`, and `Credential` are implemented as pure Java objects. They serve as the fundamental data structures of our system and are entirely agnostic of any external frameworks or database schemas.
- **Domain Services**: The actual business logic and security rules are encapsulated within Domain Services. These services act as the orchestrators of the domain. For instance, they are responsible for:

- **Invariant Enforcement**: Checking password complexity requirements (e.g., minimum length and special characters) during the user registration process.
- **Security Management**: Managing the encryption lifecycle of a Credential. The services ensure that sensitive data is correctly encrypted or decrypted before it leaves or enters the domain boundary.

2.2 Ports and Adapters: Implementation of Boundaries

A key architectural decision was the Triple-Model Strategy, separating REST DTOs, domain models, and database entities to preserve clean boundaries. As Robert C. Martin (2018) states, elements from outer layers must not be referenced in inner layers [1]. To comply with this rule, we strictly separated the REST layer (DTOs), the domain layer (models), and the persistence layer (entities), using dedicated converter classes to translate between them.

Communication between layers is handled via ports and adapters. Driving adapters (the REST API) receive DTOs and convert them into domain models, while driven adapters implement output ports such as repositories using Hibernate JPA entities. This ensures that infrastructure details do not leak into the domain core.

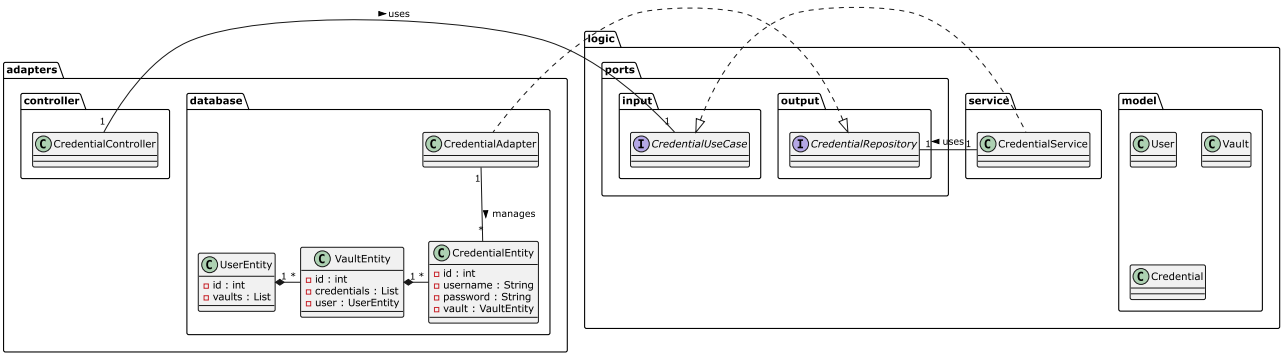

Figure 1: Hexagonal Architecture Example for Credential generated with PlantUML as SVG

2.3 Infrastructure Components and Framework Isolation

We treat all technical dependencies as infrastructure, kept strictly outside the hexagon. This includes our Persistence Stack (H2 with Hibernate JPA), ensuring the system is ready to switch to other Object-Relational Mapping (ORM) frameworks or databases with minimal impact. While we use Quarkus and RESTEasy for dependency injection and HTTP handling, these are restricted to the infrastructure and adapter layers, keeping the business logic portable.

2.4 Reflection on Challenges

The main challenge was not implementing the mapper classes, but deciding where mapping should occur within the hexagonal architecture. It was also initially unclear why multiple models for the same concept (domain, entity, DTO) were necessary, as they produce a lot of boilerplate code.

However, this separation keeps the core logic fully shielded. We learned that the cost of mapping is a small price to pay for long-term maintainability, allowing independent changes to the database or API without side effects. It also made testing individual components much easier.

3 API

The project implements the REST architectural style using JAX-RS (part of the Jakarta EE standard) on the Quarkus framework through the RESTEasy implementation. To achieve REST's goals, we consistently considered the REST constraints and implemented them using the mechanisms provided by JAX-RS.

Regarding the Uniform Interface, we designed the system around three central resources: `User`, `Vault`, and `Credential`. These resources are addressed via standard HTTP methods and uniquely

identified by artificial primary keys. All representations are exchanged in JSON format. To ensure self-descriptive messages, we introduced operation-specific DTOs, such as separate objects for creation, update, and retrieval. This guarantees that each request and response contains exactly the information required for its purpose. The interfaces are further generated and documented via OpenAPI [2]. In addition, we implemented HATEOAS using HTTP headers such as Location and Link headers, following RFC 5988 with absolute URLs, meaningful relation types, and correct media types. We also support filtering and pagination for collection resources to ensure scalable and efficient data retrieval while preserving a consistent and uniform interaction model.

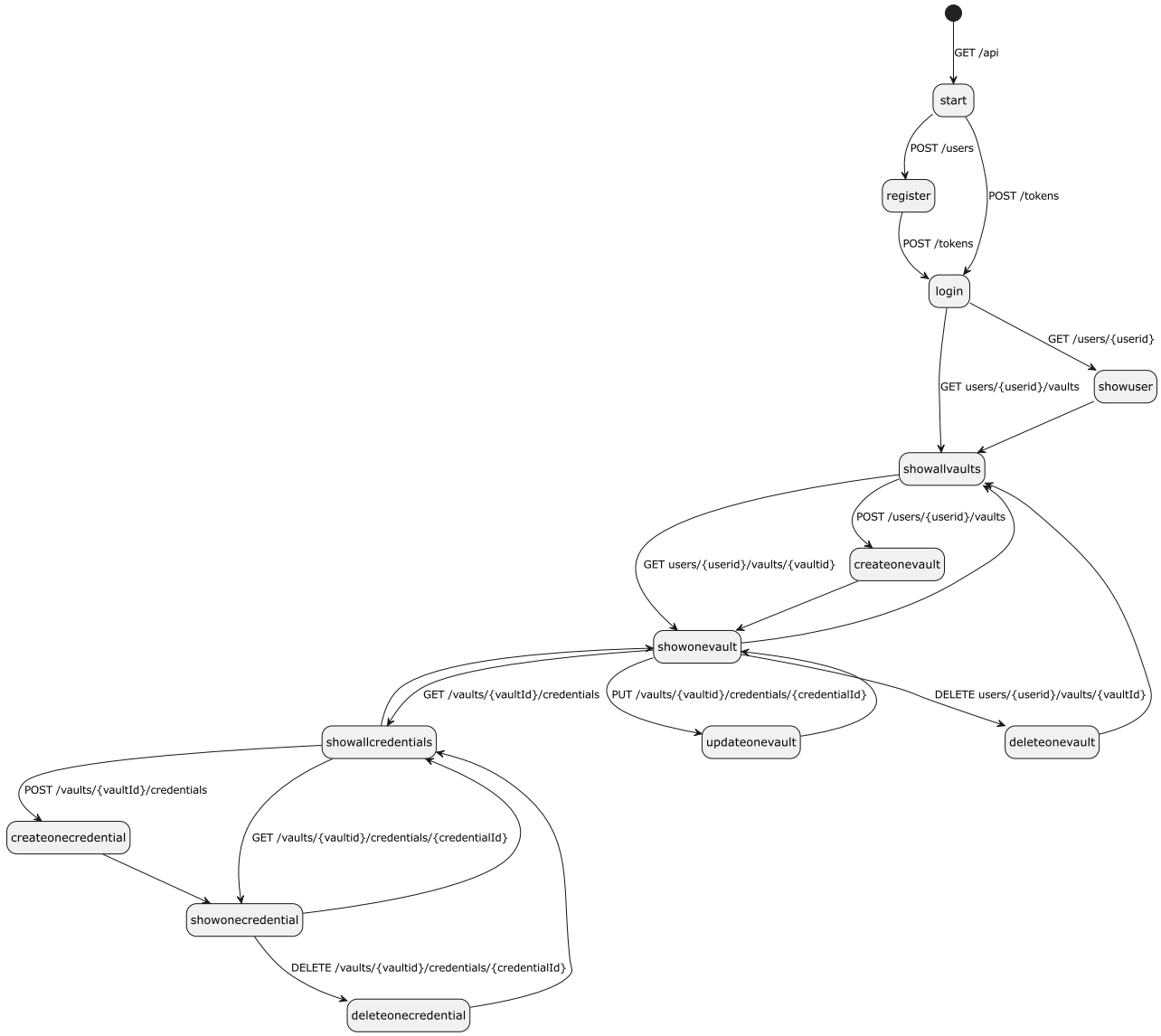

Figure 2: HATEOAS navigation state machine, generated with PlantUML as SVG

The Cacheability constraint required particular attention due to the security-sensitive nature of a password manager. We primarily rely on validation mechanisms instead of long expiration times to prevent the use of outdated data. Caching is completely disabled for credentials using Cache-Control: no-store. In less critical cases, such as retrieving vault overviews or user data, private caching with mandatory revalidation is allowed to balance performance and security. Additionally, we implemented conditional GET, PUT, and DELETE requests using ETags. By sending If-None-Match or If-Match headers, the client enables the server to detect changes, reduce unnecessary data transfer, and prevent lost updates by ensuring modifications only occur on the expected resource version.

The Layered System constraint is fulfilled through our hexagonal architecture.

The Client–Server constraint is satisfied by strictly separating responsibilities: the client handles presentation and interaction, while the server manages business logic and persistence.

The Statelessness constraint in the sense that every request contains all the information required for processing. However, in the implementation section, we explain that our encryption design introduces a server-side in-memory key store. This represents a conscious architectural trade-off and has implications for horizontal scalability.

3.1 Advantages

Using REST brings several concrete advantages to our implementation. The clear separation of concerns between client and server increases reusability, as the same backend can serve different types of clients, such as web applications, mobile apps, or desktop applications.

Our caching strategy, including validation with ETags, reduces unnecessary traffic, improves response times, and prevents lost update problems. By implementing HATEOAS, the API becomes more self-descriptive and less reliant on extensive external documentation. Clients can follow the provided links dynamically, which simplifies maintenance. Even if resource paths change, clients can adapt through hypermedia without requiring structural modifications.

Using JAX-RS with RESTEasy on Quarkus provides practical long-term benefits. All technologies are open source and actively maintained, which means regular security updates and improvements can be integrated with relatively low maintenance effort. By building on the Jakarta EE standard (JAX-RS), we avoid vendor lock-in. The application is not tightly coupled to a single framework implementation. If RESTEasy were ever discontinued, it could be replaced with another JAX-RS–compliant implementation with manageable effort, since the programming model remains the same.

3.2 Limitations

While REST with JAX-RS and Quarkus offers many advantages, there are some trade-offs. Statelessness increases network load, since each request must carry all necessary information, which can be inefficient for large or frequent operations. The REST paradigm can also be difficult for developers to fully grasp, particularly concepts like HATEOAS or proper use of hypermedia. Furthermore, JSON payloads are larger because field names are repeated in every object, adding processing and transmission overhead. While this format ensures high human-readability and abstraction, it can increase server load and infrastructure costs in large-scale systems.

4 Implementation Details

For authentication, we chose JSON Web Tokens (JWTs). Unlike Basic Authentication, the master password is only sent once at login, and a signed token is used for subsequent requests, reducing exposure of credentials. Tokens can also be invalidated individually without changing the password, improving control over compromised sessions. JWTs store metadata; we use this to include the user ID for authorization checks on each request. Quarkus handles most of the JWT workflow, simplifying issuance and verification.

The adapter mapping maps between DTOs, JSON, database entities and domain objects using manual converter classes. Initially, we encountered issues with cyclical relationships, for example a `Vault` containing a list of `Credentials` while each `Credential` also references its `Vault`. We resolved this by carefully ordering the conversion process to avoid infinite loops. These cyclical relationships provide greater flexibility in the business logic and reduce unnecessary database access.

Our business logic primarily handles password security: validating, hashing, encrypting, and securely storing credentials. We utilized the `quarkus-elytron-security-common` library, as its secure defaults for encryption and hashing significantly reduce implementation risks compared to low-level cryptographic APIs like the Java standard library. This allowed us to focus on application-specific security logic rather than low-level cryptography.

4.1 Technical Description of Encryption Workflow

During registration, the plaintext master password is transmitted to the server (protected by SSL/TLS in real-world deployment). The server first validates the password against defined security criteria. It then generates a bcrypt hash using the Quarkus Elytron security library. Bcrypt automatically includes a salt, ensuring that even leaked password hashes do not pose a practical security risk. Only the hash is stored in the database. The plaintext password is never logged or persisted.

During login, the plaintext master password is again transmitted to the server and verified against the stored bcrypt hash. If authentication succeeds, a `SecretKey` is derived from the plaintext password using the PBKDF2 key derivation algorithm. This derived key is later used to encrypt and decrypt stored credential passwords.

Initially, we considered encrypting credentials directly with the master password. However, this would require either repeatedly transmitting or storing the plaintext master password, which would introduce significant security risks. Instead, we use a derived key approach. The `SecretKey` is deterministically derived from the master password but cannot be used to reconstruct it.

A SecretKey is generated and stored in a server-side **SessionKeyStore** (implemented as an in-memory HashMap) for each authenticated JWT session. This means the key resides only in RAM and is not persisted. Even if an attacker managed to extract the `SecretKey` from memory, it would not allow reconstruction of the original master password. However, the key can still decrypt credentials that were encrypted in previous sessions, since it is derived consistently from the same master password. As mentioned in the API section on Statelessness, this introduces a controlled server-side state and therefore slightly weakens strict REST statelessness. In a horizontally scaled environment, this in-memory approach would need to be replaced by a distributed key store to ensure consistency across multiple instances.

When a new credential is created, the corresponding derived `SecretKey` is retrieved from the `SessionKeyStore` using the session ID stored as metadata in the JWT and is then used to encrypt the credential password. The session ID cannot be guessed, as it is generated as a `UUID` and securely embedded within the signed JWT.

Password-based encryption (PBE) is the approach used here. This concept is standardized by the National Institute of Standards and Technology in the United States at NIST Special Publication 800-132 [3]. Our implementation follows this concept as it suggests using a derived key from a password or passphrase, since passwords may have low entropy and are not suitable to be used directly as cryptographic keys. The derived key is then used to securely encrypt and decrypt stored user credentials.

Zero-knowledge approaches, used by systems like Bitwarden [4], ensure the server never sees the plaintext password. Our design does not fully implement this, as the master password is sent to the server before hashing or key derivation. A full zero-knowledge system would require a powerful client to perform these operations locally, shifting critical logic away from the server. This contradicts our backend-driven concept of “Backwarden”, where the core functionality resides on the backend.

5 Testing Strategy

To ensure the reliability and security of the Backwarden system, we adopted a continuous and iterative testing strategy. Instead of following a rigid top-down testing plan, we developed tests concurrently with the feature implementation. This approach ensured that every new component was immediately verified and secured against regressions before we proceeded to the next task.

5.1 Unit Tests: Logic and Mapping Isolation

For the core domain logic and utility components, we focused on isolated unit tests that run independently of infrastructure concerns. Domain service tests verify encryption and decryption flows, ensuring exceptions (e.g., invalid cryptographic states) are handled correctly. Utility components like `ValidationHelper` and `CryptoHelper` were tested to ensure password policies and AES routines work

as expected.

Additionally, we tested our converters (e.g. `CredentialDTOConverterTest`) to guarantee correct mapping between DTOs, domain models, and JPA entities, preventing information loss before data reaches the database or API.

5.2 Integration Tests: Adapters and End-to-End Flows

To verify correct interactions and overall system behavior, we implemented integration tests covering adapters, security, and REST flows.

- **Persistence Tests:** Using `@QuarkusTest` with an in-memory H2 database, we verify database interactions. `@Transactional` ensures CRUD operations and cascading deletions (e.g., deleting a `Vault` removes its `Credentials`) work correctly.
- **Security and Authorization:** Tests with the `RestAssured` library confirm logical data isolation, requiring that users cannot access others‘ resources (HTTP 403) and that invalid or missing JWTs are rejected.
- **HATEOAS Navigation:** The `HateoasNavigationTest` is a key highlight. It navigates the API purely by following hypermedia links, demonstrating that the API is fully discoverable, self-descriptive, and strictly adheres to REST principles.

5.3 Reflection on Effectiveness

Our iterative testing approach proved highly beneficial for team collaboration. Writing tests concurrently with features ensured that individual implementations were robust before merging, resulting in minimal conflicts and high code stability when integrating our work.

However, testing always involves a trade-off between effort and benefit. Our approach led to a higher proportion of integration tests and sometimes made testing complex components more challenging when designed retroactively. In future projects, we would focus on finding the right balance between strict Test Driven Development (TDD) and a practical approach, defining critical test scenarios early while keeping the overall effort manageable.

6 Learning Outcomes and Reflection

6.1 Technical Learning and Architectural Impact

Through the strict Separation of Concerns, we learned how to protect business logic from external changes. A key takeaway was our Design regarding HATEOAS workflow diagrams that had to be revised as the complexity of vault and credential states grew, teaching us that architectural details must remain flexible. Furthermore, Continuous Testing ensured high confidence in our security-critical components by integrating tests directly into the development workflow.

6.2 Team Collaboration and Workflow

Our collaboration utilized a structured Git workflow with merge requests and issue tracking. To bridge varying levels of coding experience, we invested heavily in Pair Programming early on. While time-consuming at first, it ensured every member understood the core architecture and allowed for efficient task distribution later. Starting early and maintaining a Steady Progress prevented an end-of-semester rush and allowed for thorough validation of features.

6.3 Challenges and Future Improvements

We identified bottlenecks such as Merge Complexity, where branches staying open too long led to extensive "merge hell". In the future, we would adopt Trunk-Based Development with smaller, more frequent merges. Regarding Architectural Overhead, the manual mapping boilerplate was substantial. For larger applications, we would consider automated mapping frameworks to mitigate this.

In summary, the project was a success in terms of technical deliverables and team resilience. The combination of a robust architecture and a supportive knowledge-sharing culture enabled us to build a secure, scalable RESTful backend that meets modern standards.

This article was drafted and refined using GPT-5 based on an outline containing related information. The GPT-5 output was reviewed, revised, and enhanced with additional content to improve its quality. It was then edited for improved readability and active tense, partially using Grammarly.

References

[1] R. C. Martin, *Clean Architecture: Das Praxis-Handbuch für professionelles Softwaredesign.* mitp Business, 2018.

[2] Simon Hümmer, Luke Behrsing, Christian Thorwarth, "Backwaren api – openapi specification." `https://app.swaggerhub.com/apis/thws/Backwaren/1.0.0`, 2026. Accessed: 2026-02-13.

[3] M. S. Turan, E. Barker, W. Burr, and L. Chen, "Recommendation for password-based key derivation, part 1: Storage applications," Tech. Rep. NIST Special Publication 800-132, National Institute of Standards and Technology (NIST), Gaithersburg, MD, December 2010.

[4] Bitwarden Inc., "What is zero-knowledge encryption?." `https://bitwarden.com/resources/zero-knowledge-encryption/`, 2024. Accessed: 2026-02-12.